



## Actividad de Aprendizaje Semana 14

### Indicaciones Generales:

1. El plagio se sanciona con la suspensión o expulsión del estudiante de la Universidad. Reglamento General de Estudios.
2. Sea cuidadoso con su redacción, la cual formará parte de su calificación (aspectos léxicos, sintácticos, semánticos).
3. Lea bien la pregunta o enunciado antes de proceder a su desarrollo, administre su tiempo eficazmente.

En el módulo de planificación de procesos de un sistema operativo, es necesario organizar una cola de ejecución de procesos que permita las siguientes operaciones de forma eficiente:

- Insertar un nuevo proceso con su prioridad (menor número = mayor prioridad) y su tiempo estimado de ejecución.
- Consultar cuántos procesos tienen una prioridad menor a un valor dado.
- Encontrar el proceso con menor tiempo estimado dentro de un rango de prioridades.
- Obtener la suma de tiempos estimados de todos los procesos cuya prioridad esté dentro de un rango definido.

Para ello, implemente un Treap aumentado, donde:

- Cada nodo representa un proceso con:
  - pid: Identificador del proceso.
  - prioridad: Valor entero definido por el sistema operativo.
  - tiempo: Tiempo estimado de ejecución en milisegundos.
- El Treap usará como clave principal la prioridad, y como priority del heap un valor aleatorio para mantener el balance.
- Se deben incluir campos aumentados por nodo:
  - size: número de procesos en el subárbol.
  - sumTime: suma total de los tiempos en el subárbol.

Actividades a desarrollar

#### 1. Diseño de la estructura del nodo del Treap

- Implemente una clase o estructura que contenga los campos: pid, prioridad, tiempo, priority (aleatoria), size, sumTime, left, right.
- Justifique el uso de priority y los campos aumentados.

#### 2. Inserción de procesos

- Implemente la operación insert() para insertar un nuevo proceso, manteniendo las propiedades de BST por prioridad y Max-Heap por priority.



- Asegure que se actualicen correctamente los campos size y sumTime en cada rotación.
- 3. Rotaciones (izquierda y derecha)**
  - Implemente las rotaciones necesarias para mantener el balance.
  - Verifique que las rotaciones actualicen los campos aumentados.
- 4. Consulta: contar procesos con prioridad menor a un valor dado**
  - Implemente countLessThan(p) utilizando el campo size.
- 5. Consulta: suma de tiempos de procesos entre prioridades p1 y p2**
  - Implemente una función que devuelva sumTime acumulado en un rango de prioridades.
- 6. Consulta adicional: proceso con menor tiempo estimado entre prioridades p1 y p2**
  - Recorra el Treap para encontrar el nodo con min(tiempo) cumpliendo el rango indicado. (No se requiere campo aumentado adicional, pero sí una búsqueda eficiente).
- 7. Visualización**
  - Implemente una función que imprima en orden ascendente de prioridad los procesos con todos sus datos: pid, prioridad, tiempo, size, sumTime.
- 8. Función principal (main())**
  - Simule el ingreso de al menos 10 procesos con prioridades y tiempos variados.
  - Demuestre las funciones implementadas con resultados reales.

**Responda las siguientes preguntas:**

- ¿Por qué es útil combinar un BST y un heap (como en los Treaps) en la gestión de procesos?
- ¿Qué ventaja ofrece el uso de campos aumentados frente a una implementación sin ellos?
- ¿Qué otro tipo de problemas en sistemas operativos podrían beneficiarse de este enfoque?



## Pseudocódigo

### Estructura del nodo

```
Estructura Nodo:
  pid: entero
  prioridad: entero           // clave BST
  tiempo: entero             // tiempo estimado de ejecución
  prioridadHeap: entero      // prioridad aleatoria para balance
  size: entero                // cantidad de nodos en subárbol
  sumTime: entero            // suma de tiempos en subárbol
  izq: Nodo
  der: Nodo
```

### Función para actualizar los campos aumentados

```
Función actualizar(nodo):
  Si nodo es NULL:
    retornar
  nodo.size ← 1 + tamaño(nodo.izq) + tamaño(nodo.der)
  nodo.sumTime ← nodo.tiempo + suma(nodo.izq) + suma(nodo.der)
```

```
Función tamaño(nodo):
  Si nodo es NULL: retornar 0
  retornar nodo.size
```

```
Función suma(nodo):
  Si nodo es NULL: retornar 0
  retornar nodo.sumTime
```

### Rotaciones

```
Función rotarDerecha(y):
  x ← y.izq
  T2 ← x.der

  x.der ← y
  y.izq ← T2

  actualizar(y)
  actualizar(x)

  retornar x
```



Función rotarIzquierda(x):

```
y ← x.der  
T2 ← y.izq
```

```
y.izq ← x  
x.der ← T2
```

```
actualizar(x)  
actualizar(y)
```

```
retornar y
```

### *Inserción de un proceso*

Función insertar(nodo, pid, prioridad, tiempo):

Si nodo es NULL:

```
nuevo ← nuevo Nodo  
nuevo.pid ← pid  
nuevo.prioridad ← prioridad  
nuevo.tiempo ← tiempo  
nuevo.prioridadHeap ← aleatorio()  
nuevo.size ← 1  
nuevo.sumTime ← tiempo  
retornar nuevo
```

Si prioridad < nodo.prioridad:

```
nodo.izq ← insertar(nodo.izq, pid, prioridad, tiempo)  
Si nodo.izq.prioridadHeap > nodo.prioridadHeap:  
nodo ← rotarDerecha(nodo)
```

Sino:

```
nodo.der ← insertar(nodo.der, pid, prioridad, tiempo)  
Si nodo.der.prioridadHeap > nodo.prioridadHeap:  
nodo ← rotarIzquierda(nodo)
```

```
actualizar(nodo)  
retornar nodo
```

### *Contar procesos con prioridad < p*

Función contarMenor(nodo, p):

Si nodo es NULL:

```
retornar 0
```

Si  $p \leq \text{nodo.prioridad}$ :

```
retornar contarMenor(nodo.izq, p)
```

Sino:

```
return 1 + tamaño(nodo.izq) + contarMenor(nodo.der, p)
```



### *Suma de tiempos entre p1 y p2*

```
Función sumaEnRango(nodo, p1, p2):  
  Si nodo es NULL:  
    retornar 0  
  
  Si nodo.prioridad < p1:  
    retornar sumaEnRango(nodo.der, p1, p2)  
  Si nodo.prioridad > p2:  
    retornar sumaEnRango(nodo.izq, p1, p2)  
  
  retorno ← nodo.tiempo  
  retorno ← retorno + sumaEnRango(nodo.izq, p1, p2)  
  retorno ← retorno + sumaEnRango(nodo.der, p1, p2)  
  retornar retorno
```

### *Proceso con menor tiempo en rango p1 a p2*

```
Función menorTiempoEnRango(nodo, p1, p2):  
  Si nodo es NULL:  
    retornar infinito  
  
  menor ← infinito  
  
  Si  $p1 \leq \text{nodo.prioridad} \leq p2$ :  
    menor ← nodo.tiempo  
  
  menorIzq ← menorTiempoEnRango(nodo.izq, p1, p2)  
  menorDer ← menorTiempoEnRango(nodo.der, p1, p2)  
  
  retornar mínimo(menor, menorIzq, menorDer)
```

### *Recorrido / impresión*

```
Función imprimirInorden(nodo):  
  Si nodo es NULL:  
    retornar  
  imprimirInorden(nodo.izq)  
  imprimir(pid, prioridad, tiempo, size, sumTime)  
  imprimirInorden(nodo.der)
```



*main*

```
raiz ← NULL
raiz ← insertar(raiz, 101, 15, 120)
raiz ← insertar(raiz, 102, 10, 80)
raiz ← insertar(raiz, 103, 20, 60)
// ... otros procesos

imprimirInorden(raiz)
imprimir(contarMenor(raiz, 18))           // procesos con prioridad < 18
imprimir(sumaEnRango(raiz, 10, 20))      // suma de tiempos entre prioridades
imprimir(menorTiempoEnRango(raiz, 10, 15)) // menor tiempo en ese rango
```